DATE : / /
PAGE NO. :

Assignment - 1

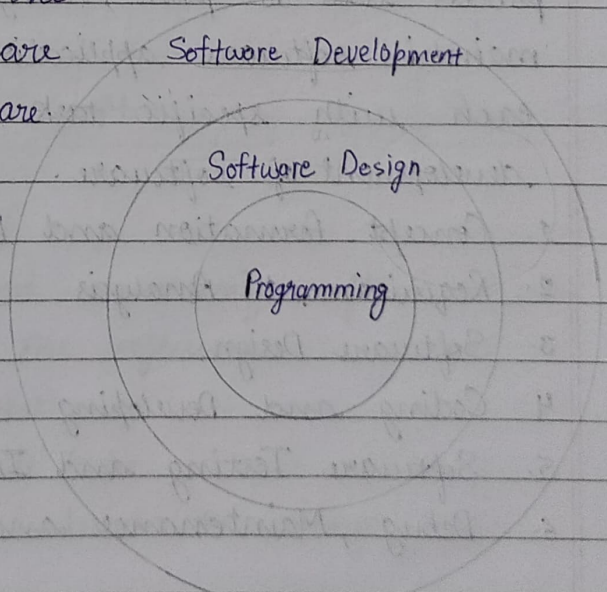
Section - A

1 What is Software Engineering ?

→ Software Engineering is the process of designing, developing, testing, and maintaining software. It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software. Software Engineering includes a variety of techniques, tools and methodologies, including requirements analysis, design, testing and maintenance. It encompasses techniques and procedures often regulated by the system development process with the purpose of improving reliability and the care.

2 Draw Software Paradigms.

→ Software paradigms refers to the methods and steps which are taken while designing software.



1. Software Development :
   - Requirement gathering
   - Software design
   - Programming
2. Software design :
   - Design
   - Maintainance
   - Programming
3. Programming :
   - Coding and Testing
   - Integration

DATE : / /
PAGE NO. :

3. Write the name of Software Component.

→ There are three components of software:
(i) Program (collection of codes)
(ii) Documentation
(iii) Operating Procedures

4. List out the Software Characteristics.

→ The Software Characteristics are:
(i) Functionality
(ii) Reliability
(iii) Efficiency
(iv) Usability
(v) Maintainability
(vi) Portability

5. Define SDLC in short.

→ The Software Development Life Cycle (SDLC) is a systematic process used to plan, design, create, test, deploy and maintain software application. It consists of phases, each with specific task, to ensure successful development of software.

1. Concept formation and Planning
2. Requirement Analysis
3. Software Design
4. Coding and Developing
5. Software Testing and Integration
6. Debug, Maintenance and Support.

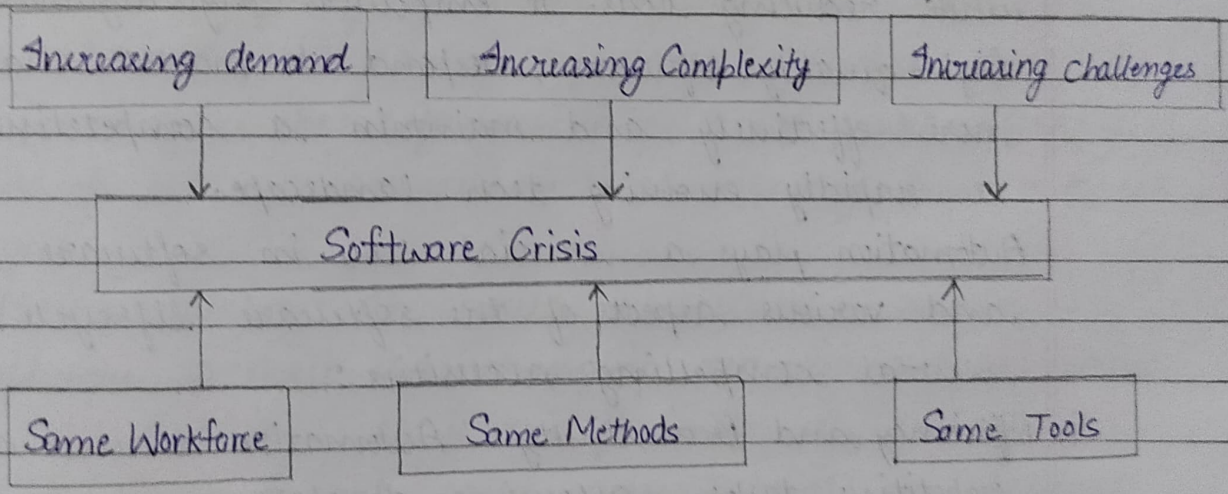
DATE : / /
PAGE NO. :

Section - B

1. Explain Software Crisis.

→ Software Crisis is a term used in computer Science for the difficulty of writing useful and efficient computer programs in the required time.

If we will use the same workforce, same methods and same tools after the fast increase in software demand, software complexity and software challenges, then there arise some problems like software budget problems, software efficiency problems, software management and delivery problem etc. This condition is called a Software Crisis.



Causes of Software Crisis:

(i) The cost of owning and maintaining software was as expensive as developing the software
(ii) Software is very inefficient
(iii) The quality of software is low.
(iv) The software complexity is harder to change.

factors contributing to software Crisis:

(i) Poor project management
(ii) Lack of adequate training in software engineering.

1

(iii) Less skilled project members.
(iv) Low productivity improvements.

Solution of Software Crisis:
(i) Reduction in software over budget
(ii) The quality of the software must be high
(iii) Less time is needed for a software project.
(iv) Software must meet user requirements.

2. Explain the necessity of automation in software.
→ Automation in software development is necessary because it enhances efficiency, quality, speed and reliability while reducing costs. It empowers organizations to deliver high-quality software, respond to market changes more effectively and maintain a competetive edge in a rapidly evolving tech landscape.
Automation plays a crucial role in software development and various aspects of the software lifecycle due to several compelling necessities:
(i) Efficiency and Productivity: Automation reduces manual, repetitive tasks, allowing developers and teams to focus on more creative and complex aspects of software development. This results in faster development cycles and increased productivity.
(ii) Consistency: Automated processes ensure that tasks are performed consistently without human errors.
(iii) Quality Assurance: Automation can conduct repetitive testing, code analysis and quality checks more thoroughly and quickly than humans. This leads to higher software quality and fewer bugs in final product.

DATE : / /
PAGE NO. :

(iv) Continuous Integration and Continuous Development : Automation enables the integration and development of code changes to production enviroments reliably and continuously.

(v) Scalability : Automation allows software development and deployment processes to scale efficiently as project size grow.

(vi) Security : Automated security scanning tools can identify vulnerabilities and security issues in the code and dependencies allowing for timely remediation and threat mitigation.

3. Derive the Enginering Aspects of Software production.

→ The Engineering aspects of software production are :

(i) Efficiency : The software should not make wasteful use of system resources such as memory and processor cycle.

(ii) Maintainability : It should be possible to evolve the software to meet the changing requirements of users.

(iii) Dependability : It is the flexibility of the software that ought to not cause any physical or economic injury within the event of system failure. It includes a range of characteristics such as reliability, security, and safety.

(iv) Functionality : The software system should exibhit the proper functionality i.e. it should perform all the functions it is supposed to perform.

(v) Adaptability : The software system should have the ability to get adapted to a reasonable extent with the changing requirement.

(vi) In Time : Software should be developed well in time.

1

DATE : / /
PAGE NO. :

(vii) Within budget: The software development costs should not overrun and it should be within the budgetary limit.

Section-C

1 Discuss about the job responsibilities of programmers and software Engineers as a Software developers.

→ The job responsibilities of programmers and software engineer includes working on engineering principles for software development and making modifications to an ongoing project (in terms of architecture, design or testing), testing also includes UAT (user acceptance testing).

Job responsibilities of a software developer are:

- Coordinate with the technical director on current programming tasks.
- Collaborate with other programmers to design and implement features.
- Quickly produce well-organized, ~~opit~~ optimized and documented source code.
- Create and document software tools required by artists or other developers.
- Debug existing source code and polish feature sets.
- Contribute to technical design documentation.
- Work independently when required.
- Continuously learn and improve skills.
- Attention to detail is essential and all tasks must be carried out to the highest standard.
- Develop solutions by gathering information, feedback from users, the case study of system flows, and overall processess.

DATE : / /
PAGE NO. :

- Develop information systems by developing, designing, maintaining and installing software solutions
- Should be familiar with SDLC (Software Development Life cycle)
- Should be able to deliver documentation and demonstrate different solutions for developing flowcharts, code comments and layouts
- Deliver and meet the standards for engineering structures and handle complexity to maintain the flow.
- Maintain the privacy of confidential pieces related to any project.

2 Give the briefly statement about Software Characteristics.

→ Software engineering is the process of designing, developing and maintaining software systems. A good software is one that meets the needs of its users, performs its intended functions reliably and is easy to maintain. There are several characteristics of good software that are commonly recognized by software engineers, which are important to consider when developing a software system. These characteristics include functionality, usability, reliability, performance, security, maintainability, reusability, scalability and testability.

1. Functionality: The software meets the requirements and specifications that it was designed for, and it behaves as expected when it is used in its intended enviroment.

2. Usability: The software is easy to use and understand and it provides a positive user experience

DATE : / /
PAGE NO. :

3. Reliability : The software is free of defects and it performs consistently and accurately under different conditions and scenarios.

4. Maintainability : The software is easy to change and update and it is well-documented, so that it can be understood and modified by other developers.

5. Efficiency : The software should not make wasteful use of system resources such as memory and processor cycle.

6. Portability : It is the possibility to use the same software in different enviroments. It applies to the software that is available for two or more different platforms or can be recompiled for them.

7. Performance : The software runs efficiently and quickly and it can handle large amounts of data or traffic.

8. Security - The software is protected against unauthorized access and it keeps the data and functions safe from malicious attacks.

9. Reusability - The software can be reused in other projects or applications and it is designed in a way that promotes code reuse.

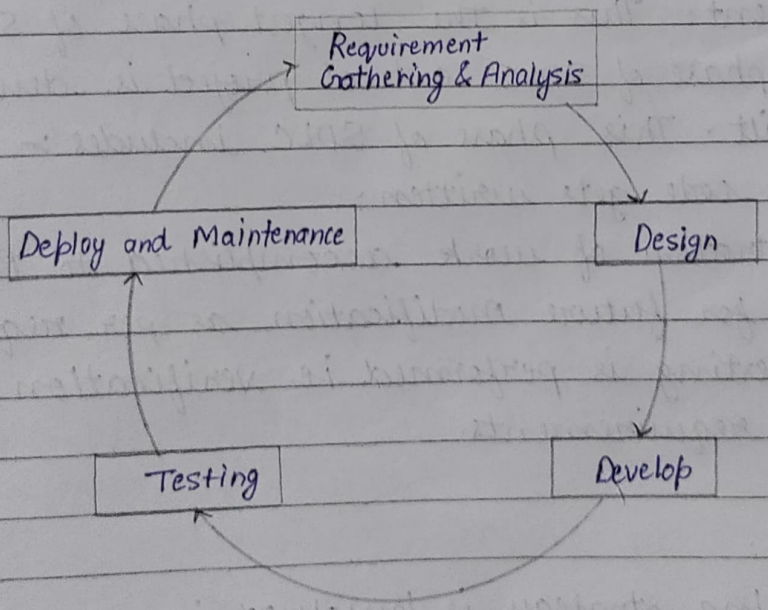
10. Scalability - The software can handle an increasing workload and it can be easily extended

to meet the changing requirements.

11. Testability: The software is designed in a way that makes it easy to test and validate and it has a comprehensive test coverage.

3. Explain the Software Development Life Cycle with suitable diagram.

→ Software Development Life Cycle (SDLC):
It is a process that describes how to develop, design and maintain the software project ensuring that all the functional and user requirement, goals and objective are met. This methodology improves the quality of the software project and over all process of software development.



Software Development Life Cycle (SDLC)

DATE : / /
PAGE NO. :

Phases of Software Development life Cycle :

1. Requirement Analysis : Explicit and implicit software requirement specification obtained by studying problem domain, operational timelines and procedure enviroment.
SRS (Software Requirement Specification) is prepared in this phase only.

2. Design Specification :
- Software development plan, implicit and explicit Software requirement and the software architecture.
- Internal design : Algorithm & pseudo code
- External design : User interface with interface, global and control structure.

3. Development : This is the longest phase of SDLC as in this phase of SDLC actual project is developed and built. This phase of SDLC includes :-
- Actual code gets written.
- Demonstration of work accomplished to Business analyst for future modification as per requirement.
- Unit testing is performed i.e. verification of code as per requirements.

4. Testing :
- The testing strategy is involved in almost all stages of SDLC. However this phase of SDLC refers to the only testing of system where bugs / defect of the system are reported, tracked and fixed. The system / project is migrated to a test enviroment

DATE : / /
PAGE NO. :

and different type of testing is performed like functional, integration, system and acceptance. This is performed until the project reaches the quality standards as specified in SRS. This phase of SDLC includes : (i) System is tested as whole. (ii) Different type of testing to report and fix bugs.

5. Deployment and Maintenance :

- In this phase of SDLC, once the system is tested, it is ready to go live. The system may be first released for limited users, and tested in a real business enviroment for UAT (User Acceptance Testing). This phase of SDLC includes system is ready to be delivered.
- The system is installed and put into use.
- Correction of errors that were not caught before.
- Improving system and ultimately enhanced system in data center.